

PRISM: Precision Robustness in Spectral vs. Additive Modulation for Post-Training Quantization of LLM Adapters

Nicolás Andrey Orozco Giraldo*, Luis Eduardo Muñoz Guerrero*, Yony Fernando Ceballos†

*Universidad Tecnológica de Pereira, Pereira, Colombia
nicolas.orozco@utp.edu.co, lemunozg@utp.edu.co

†Facultad de Ingeniería, Grupo de Ingeniería y Sociedad, Universidad de Antioquia, Medellín, Colombia
yony.cebillos@udea.edu.co

Abstract—Not every parameter-efficient adapter can be fully folded into a model’s frozen weights before deployment. LoRA’s entire learned update lives in a low-rank matrix that merges directly into the weight it adapts. SVMO, one of three coupled operators in the S^3 adapter, modulates a weight matrix’s singular values and is the only one of the three that produces a mergeable weight delta at all; the other two, a continuous-depth flow and a cross-attention bridge, operate on activations at inference time and are never merged or quantized regardless of what happens to the backbone. On their own, without those other two operators, SVMO’s merge does almost nothing: on a held-out set where the frozen base model scores 8.42 perplexity, merging SVMO in moves it to 8.39, while merging LoRA moves it to 5.72. SVMO’s update is also $47\times$ smaller than LoRA’s in Frobenius norm. Both facts might suggest SVMO’s merge is the safer one to quantize: smaller change, smaller stakes. Measured post-training quantization says otherwise. At 4-bit symmetric per-channel quantization, the SVMO-merged model’s perplexity rises by 5.17 points and LoRA’s by 4.36, the opposite of what update size and task contribution alone would predict. A magnitude-matched control, rescaling LoRA’s update to exactly match SVMO’s Frobenius norm before quantizing, reproduces this ordering: at equal magnitude, LoRA disturbs the merged weights’ per-channel statistics less than SVMO does. For adapters like S^3 , where most of the learned behavior lives in components that never touch the frozen weights, the small remainder that does get merged can still be the more fragile part to quantize, regardless of how little it contributes to task performance or how much smaller its update is.

I. INTRODUCTION

Fine-tuned language models rarely run in the precision they were trained in. After training, the adapter is merged into the frozen base weights, and the resulting model is quantized for deployment: to 8-bit or 4-bit integers, for a GPU with less memory than training used, or for a CPU or phone with none of the accelerator support training assumed. This merge-then-quantize pipeline is standard practice. Whether a specific parameter-efficient fine-tuning (PEFT) method makes that second step easier or harder is rarely examined; the adapter’s job is usually judged to end at the accuracy it reaches during evaluation, before quantization enters the picture.

Not every adapter can be folded into the frozen weights the same way. LoRA’s entire learned effect lives in a low-rank matrix BA that merges directly into W ; quantizing the merged

model quantizes all of what LoRA learned. S^3 couples three operators, and only one of them touches W at all: SVMO modulates W ’s existing singular values through a bounded, saturating nonlinearity, rescaling a direction W already has by at most a fixed fraction and never introducing one it does not have. The other two, defined formally in Section III, act on activations at inference time rather than on W itself; neither produces a weight delta, and neither is touched by quantizing the backbone, merged or not.

On its own, without NMF and STB, SVMO’s merge does very little. On the held-out set used throughout this paper, it moves the frozen base model’s perplexity from 8.42 to 8.39; LoRA moves the same base model to 5.72. This asymmetry is not an artifact of this checkpoint: S^3 ’s own evaluation reports the same pattern, that SVMO without STB contributes almost nothing on its own [?], because most of the adapter’s benefit comes from the two operators that quantization can never touch anyway. The question this paper asks is narrower than comparing two adapters that do the same job. Of the two things that can actually be merged and quantized, LoRA’s full update and SVMO’s small remainder, does either one’s geometry make it more fragile under quantization, independent of how much each contributes to the model’s accuracy on its own?

A substantial literature studies quantization and LoRA together, but almost entirely in the opposite order: the base model is quantized first and LoRA is trained on top of it [?], quantization is folded into training itself [?], or the small adapter matrices are what gets quantized [?]. Merging a trained adapter into full-precision weights first and quantizing the merged result for deployment second is the ordinary path outside the quantization literature, and whether an adapter’s own geometry affects how well that merge survives quantization is the question Section II returns to.

It is tempting to answer that question by comparing the merged weights directly: whichever adapter changes W less should be the safer bet under quantization, since quantization error grows with the dynamic range and heavy-tailedness of a weight matrix’s entries. Applied to the real checkpoints above, this comparison has a clean answer: SVMO’s update is

$47\times$ smaller in Frobenius norm, and it leaves the per-channel kurtosis of the merged weights essentially where it started, while LoRA’s shifts it by up to three orders of magnitude in the largest MLP projections. SVMO looks like the obviously safer choice, on top of already being the one that changes the model’s behavior less.

That comparison is confounded, and the confound is fixable. LoRA’s larger footprint could reflect something about how additive, unconstrained updates behave in general, or it could simply reflect that reaching a much larger change in the model’s output took a much larger change to the weights. Separating those two explanations only requires rescaling LoRA’s update, per matrix, to the same Frobenius norm as SVMO’s, and re-measuring. When we do this, the ranking reverses: at matched magnitude, LoRA’s update disturbs per-channel weight statistics less than SVMO’s does. The first comparison mistook update size for update safety.

The real test is whether this magnitude-controlled ranking predicts what happens under actual post-training quantization of the original checkpoints, before any artificial rescaling. It does. At 8-bit symmetric per-channel quantization, both adapters lose essentially nothing: the held-out perplexity change is within noise of zero for either. At 4-bit, where quantization error is large enough to matter, LoRA’s merged model degrades by 4.36 perplexity points and SVMO’s by 5.17, the opposite order from the naive comparison and the same order the magnitude-controlled experiment predicted in advance.

This paper reports that result and the two experiments behind it, in the order they were run: the naive comparison, the controlled experiment that overturns it, and the real quantization measurement that confirms the controlled experiment. Our contributions are:

- A demonstration that ranking PEFT adapters by the raw magnitude of their merged weight update, already common practice, can rank them backwards relative to how they behave under real post-training quantization.
- A magnitude-matched control that isolates the effect of an adapter’s update *geometry* (additive low-rank vs. bounded spectral rescaling) from its update *size*, and correctly predicts which adapter degrades less under real 4-bit quantization before that quantization is ever run.
- Measured perplexity change for both adapters at two quantization precisions, 8-bit and 4-bit, on real checkpoints of the same 7B base model: negligible for either adapter at 8-bit, and substantial and correctly ordered by the magnitude-controlled prediction at 4-bit.

II. RELATED WORK

A. Weight Outliers and Post-Training Quantization

Naive uniform quantization loses the most precision on the weights or activations that stray furthest from the bulk of the distribution. GPTQ and its successors reduced this loss by calibrating per-channel scales against a small dataset, but even calibrated quantizers struggle when a distribution

has a few entries orders of magnitude larger than the rest. AWQ [?] addresses this directly: it identifies the weight channels most sensitive to activation outliers and protects them with a channel-wise scaling factor before quantizing, rather than treating every channel identically. SpinQuant [?] approaches the same problem differently, learning a rotation matrix that redistributes a weight matrix’s energy away from extreme outliers before any rounding happens. Both methods exist because how spiky a matrix’s entries are, not only how large the matrix is, determines how much a low-bit quantizer damages it. The same concern leads GGUF’s Q4_K_M, the dominant 4-bit format for CPU and phone deployment via `llama.cpp`, to keep outlier-sensitive portions of a tensor at higher precision rather than quantizing every block uniformly. Our kurtosis and max-to-standard-deviation measurements in Section III track the same property; the question we ask is how a PEFT adapter’s merge changes that property, rather than how to correct for it afterward.

B. Quantization and Parameter-Efficient Fine-Tuning

The dominant way quantization and LoRA-style adapters meet in the literature runs in the opposite order from this paper. QLoRA [?] quantizes the frozen base model to 4-bit NormalFloat first and trains LoRA on top of the frozen, quantized weights, so the adapter learns to compensate for quantization noise that is already present during training. LoftQ [?] refines the initialization of that same pipeline: it alternates quantization and low-rank approximation to find a quantized base and an initial adapter that jointly minimize the Frobenius-norm gap to the original full-precision weight, before LoRA training even starts. QA-LoRA [?] allocates quantization in groups aligned with the adapter’s own structure, so the trained adapter can be folded back into the quantized base without a separate dequantization step. LowRA [?] moves quantization to the adapter side entirely, compressing the trained LoRA matrices A and B down to sub-2-bit precision. L4Q [?] folds quantization into the training loop itself.

Each of these methods answers a version of the question: how should we quantize, given that LoRA is also part of the picture. This paper asks a different question. Once an adapter has already been trained, merged back into full-precision weights, and judged on the accuracy that merge reaches, quantizing the result for deployment is a separate, later step, and the choice of adapter made earlier can still affect how much that later step costs. Train-then-merge-then-quantize is the ordinary path for any adapter evaluated first on accuracy and deployed only afterward; to our knowledge, no prior work compares adapters of different geometry along that specific path.

C. Spectral Adaptation

SVMO belongs to a small family of PEFT methods that adapt a frozen weight matrix through its singular value decomposition rather than through an additive term. Zhang and Pilanci’s Spectral Adapter [?] rotates or shifts the top singular vectors of W directly, with a parameter count that grows

with the matrix’s rank. SVMO, introduced as part of the S³ adapter [?], leaves U and V untouched instead and learns a bounded, pointwise nonlinear rescaling of the singular values Σ alone, through a small modulation network with $O(H^2)$ parameters independent of the matrix’s size. Values can only move by a fixed fraction of where they started. That bound is what motivates the naive expectation this paper tests, and overturns.

III. METHOD

A. The Two Mergeable Updates

Let $W \in \mathbb{R}^{d_{out} \times d_{in}}$ be a frozen weight matrix. LoRA introduces $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$, trained so the adapted layer computes $Wx + \frac{\alpha}{r}BAx$. Merging folds this into the weight directly:

$$W_{\text{LoRA}} = W + \frac{\alpha}{r}BA, \quad (1)$$

with $r = 8$ and $\alpha = 16$ in the checkpoints used here.

SVMO instead operates on W ’s singular value decomposition, computed once offline: $W = U\Sigma V^\top$, truncated to the top k components $U_k \in \mathbb{R}^{d_{out} \times k}$, $\Sigma_k = \text{diag}(\sigma_1, \dots, \sigma_k)$, $V_k \in \mathbb{R}^{k \times d_{in}}$. A two-hidden-layer MLP $g_\theta: \mathbb{R} \rightarrow \mathbb{R}$ of width $H = 32$ learns a pointwise modulation of the log-singular values,

$$\Delta\sigma_i = \sigma_i \cdot \alpha \cdot \tanh(g_\theta(\log(\sigma_i + \varepsilon))), \quad i = 1, \dots, k, \quad (2)$$

with $\alpha = 0.3$ bounding the fractional change any singular value can undergo and $\varepsilon = 10^{-8}$. Merging gives

$$W_{\text{SVMO}} = W + U_k \text{diag}(\Delta\sigma) V_k^\top. \quad (3)$$

W_{LoRA} and W_{SVMO} have exactly the shape of W and are computed once, offline, before any quantization. Everything measured in Section IV operates on these merged matrices directly, not on W , A , B , or the SVD factors separately.

B. Why NMF and STB Stay Out of Scope

NMF defines a continuous-depth flow over a hidden state h , $dh/d\tau = f_\theta(h, \tau)$, integrated with RK4 at inference time; STB is a cross-attention module coupling h ’s projection onto U_k with the modulated singular values $\Sigma + \Delta\sigma$. Neither has the form $W \rightarrow W + \Delta$. Both require a forward pass through a small network at every token, at every layer, regardless of what precision the frozen backbone is stored in. Quantizing W changes what W costs to store and multiply; it does not touch f_θ or STB’s coupling weights, which stay resident in full precision either way. That is the sense in which Section I calls SVMO the only one of the three operators quantization can reach at all.

C. Magnitude-Matched Control

For a given matrix, let Δ_{SVMO} and Δ_{LoRA} denote the two updates as trained, from Equations 3 and 1. To separate how much of LoRA’s larger footprint is geometry and how much is size, we construct a magnitude-matched counterfactual,

$$\Delta_{\text{LoRA}}^{\text{scaled}} = \frac{\|\Delta_{\text{SVMO}}\|_F}{\|\Delta_{\text{LoRA}}\|_F} \Delta_{\text{LoRA}}, \quad (4)$$

computed separately for each matrix, so that $\|\Delta_{\text{LoRA}}^{\text{scaled}}\|_F = \|\Delta_{\text{SVMO}}\|_F$ exactly.

Two properties of Equation 4 matter. First, the rescaling is isotropic: multiplying a matrix by a positive scalar changes its size, not its direction, so $\Delta_{\text{LoRA}}^{\text{scaled}}$ points exactly where the trained LoRA update points. Any change between the raw and the rescaled comparison is attributable to magnitude alone. Second, the rescaling is applied per matrix rather than once globally, because $\|\Delta_{\text{SVMO}}\|_F$ itself varies from one matrix to the next, set by that matrix’s own singular value spectrum; a single global scale would leave magnitude uncontrolled at the level Section IV actually measures.

Equation 4 produces a hypothesis, not a conclusion: if size rather than geometry explains why LoRA disturbs weight statistics more than SVMO in the raw comparison, that gap should shrink or reverse once magnitude is matched. What justifies treating the resulting prediction as more than a property of this one construction is that it is checked independently in Section IV: computed without touching a quantizer, it correctly anticipates which of the two *unscaled*, actually-trained checkpoints degrades less under real 4-bit quantization.

D. Weight-Distribution Metrics

For a merged matrix $W' \in \mathbb{R}^{d_{out} \times d_{in}}$, row i has entries $w_{i,1}, \dots, w_{i,d_{in}}$ with mean μ_i and standard deviation s_i . Two statistics are reported per matrix, each averaged over its d_{out} rows:

$$\kappa_i = \frac{1}{d_{in}} \sum_{j=1}^{d_{in}} \left(\frac{w_{i,j} - \mu_i}{s_i} \right)^4 - 3, \quad (5)$$

$$\rho_i = \max_j |w_{i,j}| / s_i, \quad (6)$$

excess kurtosis and the max-to-standard-deviation ratio. Both are zero and moderate, respectively, for a Gaussian row, and both grow when a few entries in the row sit far from the rest. They track the same property that per-channel quantizers such as GPTQ and AWQ calibrate against: a row with a small number of extreme entries needs a wider quantization range, which determines how coarsely every other entry in that row gets rounded.

E. Simulated Post-Training Quantization

Quantization is applied per output channel, matching the granularity of the metrics above. For b bits, with $q_{\max} = 2^{b-1} - 1$, each row of a merged matrix is quantized and immediately dequantized:

$$s_i = \frac{\max_j |w_{i,j}|}{q_{\max}}, \quad (7)$$

$$\hat{w}_{i,j} = s_i \cdot \text{clip}(\text{round}(w_{i,j}/s_i), -q_{\max}-1, q_{\max}). \quad (8)$$

$\hat{W}$ replaces W for every subsequent computation; the rounding happens once, offline, and all inference afterward runs in full precision on the dequantized values. This measures the effect of quantization error on model quality directly, without a low-bit compute kernel, and is standard for studying quantization sensitivity in isolation from quantization speed. We

TABLE I
RAW COMPARISON, AVERAGED OVER 196 MATRICES.

	Base	+SVMO	+LoRA
Kurtosis κ	5.6745	5.6745	5.5853
Max/std ρ	4.8799	4.8799	4.8776
$\ \Delta\ _F/\ W\ $	—	0.051%	2.400%

TABLE II
MEAN $|\Delta\kappa|$ BY MATRIX, RAW COMPARISON.

Matrix	SVMO	LoRA	Ratio
q_proj	0.00013	0.00170	13×
k_proj	0.00037	0.00264	7×
v_proj	0.00025	0.00117	5×
o_proj	0.00031	0.00121	4×
gate_proj	0.00054	0.5996	1104×
up_proj	0.00004	0.0152	390×
down_proj	0.00020	0.00370	19×

TABLE III
MEAN $|\Delta\kappa|$ BY MATRIX, MAGNITUDE MATCHED.

Matrix	SVMO	LoRA (scaled)	Ratio
q_proj	0.00013	0.00001	0.05×
k_proj	0.00037	0.00004	0.10×
v_proj	0.00025	0.00000	0.02×
o_proj	0.00031	0.00000	0.01×
gate_proj	0.00054	0.00019	0.34×
up_proj	0.00004	0.00001	0.37×
down_proj	0.00020	0.00001	0.03×

TABLE IV
MEASURED PERPLEXITY, ORIGINAL CHECKPOINTS.

	fp32	8-bit	4-bit
Base (no adapter)	8.42	—	—
+SVMO	8.39	8.39 (−0.003)	13.56 (+5.17)
+LoRA	5.72	5.73 (+0.015)	10.08 (+4.36)

use $b = 8$ as a mild baseline and $b = 4$ as the level that matters for the deployment scenario motivating this paper, matching Q4_K_M’s bit-width (Section II) even though the block structure of Equation 8 is simpler.

IV. EXPERIMENTS

A. Setup

All checkpoints come from the same training run of Qwen2.5-7B-Instruct (28 layers, hidden size 3584): SVMO with rank $k = 128$ and modulation width $H = 32$, and LoRA with $r = 8$ (Section III). Held-out perplexity is measured on the same eight examples used throughout the S³ project, the last eight instruction-response pairs of the Alpaca training split, tokenized with the project’s standard template, 1,021 scored tokens in total. Every number below, including for the frozen base model with no adapter merged at all, comes from this same held-out set, streamed one decoder layer at a time so the full forward pass runs on a CPU with no GPU resident model.

B. The Raw Comparison

On this held-out set, the frozen base model scores 8.42 perplexity. Merging SVMO moves it to 8.39; merging LoRA moves it to 5.72. Across all $28 \times 7 = 196$ adapted matrices, SVMO’s update averages 47× smaller than LoRA’s in Frobenius norm relative to $\|W\|$ (0.05% vs. 2.4%).

Table I reports the two weight-distribution statistics from Section III, averaged over all matrices. SVMO leaves both essentially where the frozen base model started; LoRA’s merge visibly shifts both.

The gap is not uniform across matrix types. Table II reports the mean absolute kurtosis shift by matrix; the largest MLP projections show the widest gap, with LoRA’s shift over three orders of magnitude larger than SVMO’s in `gate_proj`.

Taken at face value, this comparison says SVMO’s merge is the safer one to quantize: smaller in magnitude, and gentler on the statistics per-channel quantizers depend on.

C. Magnitude-Matched Control

Rescaling LoRA’s update per matrix to match SVMO’s Frobenius norm exactly (Equation 4) and repeating the same measurement reverses the ranking. At matched magnitude, LoRA disturbs kurtosis less than SVMO in 188 of 196 matrices, and the max-to-standard-deviation ratio less in 179 of 196. Table III repeats the by-matrix breakdown with LoRA’s update scaled down to SVMO’s size before measuring: the two projections that showed LoRA as over 1,000× more disruptive in the raw comparison now show SVMO as the more disruptive one instead.

D. Real Post-Training Quantization

Table IV reports measured perplexity for both adapters, at both quantization levels, on the original, unscaled checkpoints.

At 8-bit, neither adapter’s merge is disturbed enough to matter: both changes are under 0.02 perplexity points, a small fraction of the base model’s own 8.42. At 4-bit, the gap is large, and it confirms the magnitude-matched control’s prediction: SVMO’s merged model degrades more (+5.17) than LoRA’s (+4.36), despite SVMO’s update being 47× smaller and despite SVMO contributing a fraction of LoRA’s improvement over the frozen base model to begin with.